
The Model & Basic Computations

Chapter 1 and 2

The Model

Broadcast

Spanning Tree Construction

Traversal

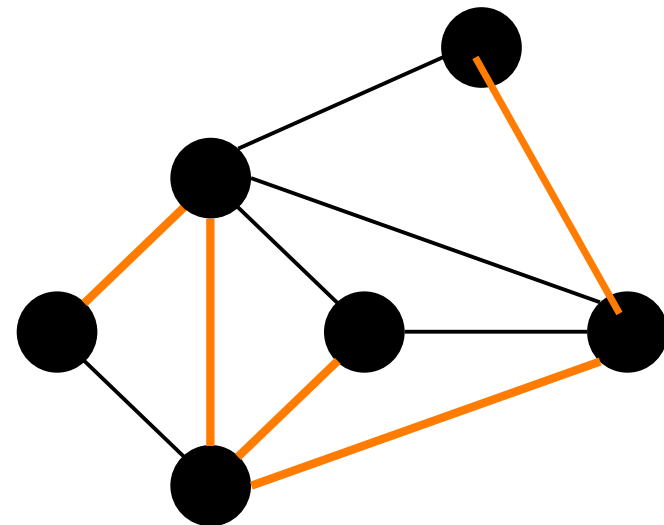
Wake-up

Spanning Tree Construction

A spanning tree T of a graph $G = (V, E)$ is an acyclic subgraph of G such that $T = (V, E')$ and $E' \subset E$.

Assumptions:

single initiator
bidirectional links
total reliability
 G connected



Motivation.

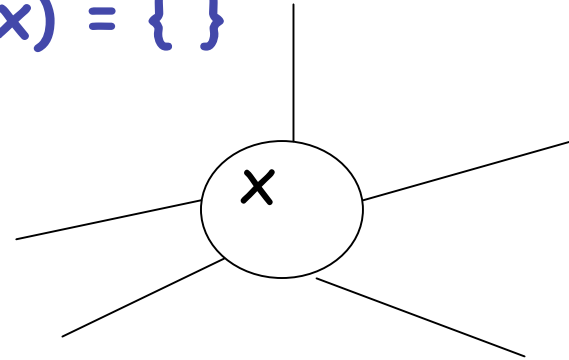
- Performing communication tasks (**broadcast**) on a subnet is more efficient (**m** is smaller).
- The subnet must be
 - + Spanning
 - + Connected

Any **Spanning Tree** is the connected spanner having minimal number of links

The Distributed ST Problem

Initial

Configuration: $\forall x, \text{Tree-neighbors}(x) = \{ \}$

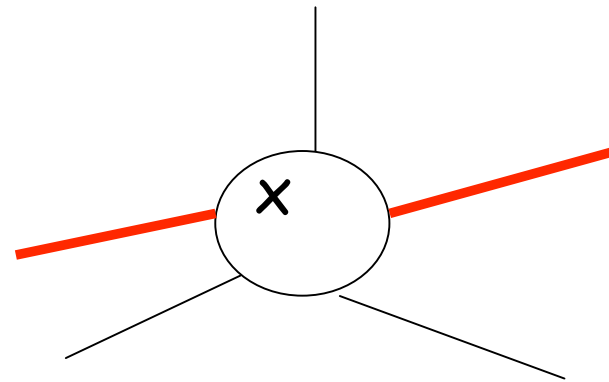


Final Configuration:

$\forall x, \text{Tree-neighbors}(x) = \{x\text{-links that belong to the ST}\}$

Note.

nodes may not know the **ST**
even **after** its computation



Protocol SHOUT (Single Initiator). The idea

How can **x** select its ST-Neighbors?

Just ask them!

Neighbors' answer:

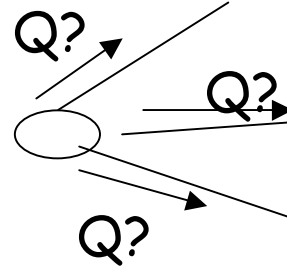
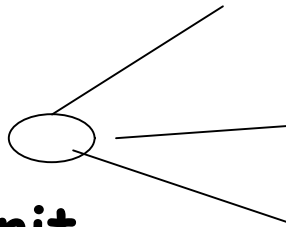
Always say **YES** to the first request
and

NO to any further ones

The **Initiator** will start asking and then the protocol proceeds as **broadcast** with **acks**

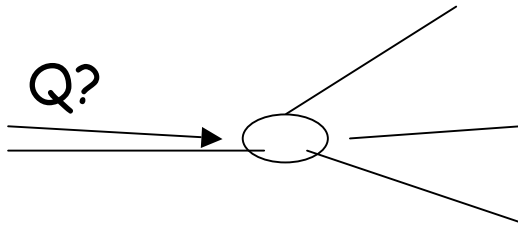
1.

Init

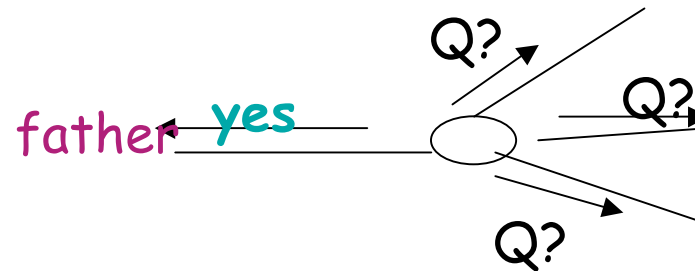


Q? = do you want to be
my neighbour
in the spanning tree ?

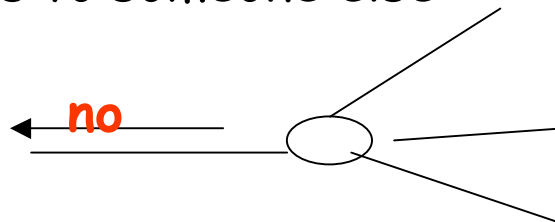
2.



If it is the **first** time:



If I have already answered
yes to someone else:



States $S = \{\text{INITIATOR}, \text{IDLE}, \text{ACTIVE}, \text{DONE}\}$

Sinit = {INITIATOR, IDLE}

Sterm = {DONE}

INITIATOR

Spontaneously

root := true

Tree-neighbours := { }

send(Q) to N(x)

counter := 0

become ACTIVE

IDLE

receiving(Q)

root := false

parent := sender

Tree-neighbours := {sender}

send(yes) to sender

counter := 1

if counter = |N(x)| then

become DONE

else

send(Q) to N(x) - {sender}

become ACTIVE

ACTIVE

receiving(Q)

send(no) to sender

receiving(yes)

Tree-neighbours:=

Tree-neighbours $\cup$ sender

counter := counter +1

if counter = $|N(x)|$

become DONE

receiving(no)

counter := counter +1

if counter = $|N(x)|$

become DONE

Note:

SHOUT = FLOOD + REPLY

TERMINATION of SHOUT.

-SHOUT = FLOOD(Q) --> Every $\times$ (but Init) will receive the Q? msg and will answer YES/NO to each Q?

This implies every node eventually goes to status DONE. --> local termination

CORRECTNESS of SHOUT

- If x is in Tree-neighbours of y , y is in Tree-neighbours of x
(Exercise 1)
- If x sends YES to y , then x is in Tree-neighbour of y
and is connected to the initiator by a chain of YES-links
(Exercise 2)
- Every x (except the initiator) sends exactly one YES
--> (no cycles!!)



The spanning graph defined by the Tree-neighbour relation is **connected**, **acyclic** and contains **all** nodes

Message Complexity

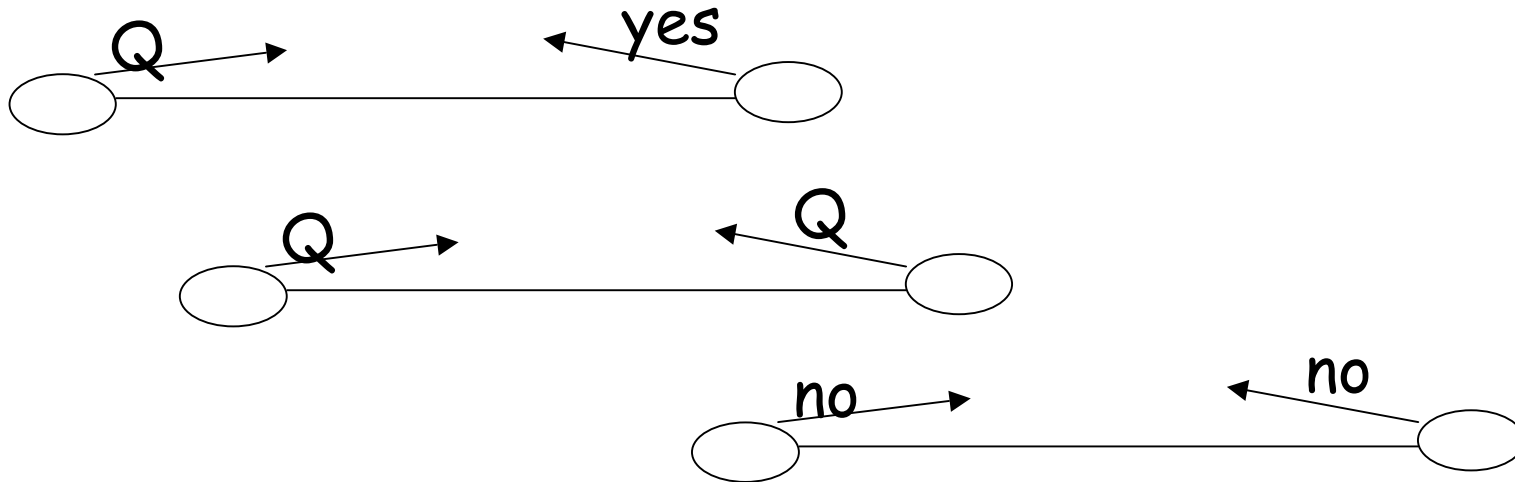
SHOUT = FLOOD + REPLY



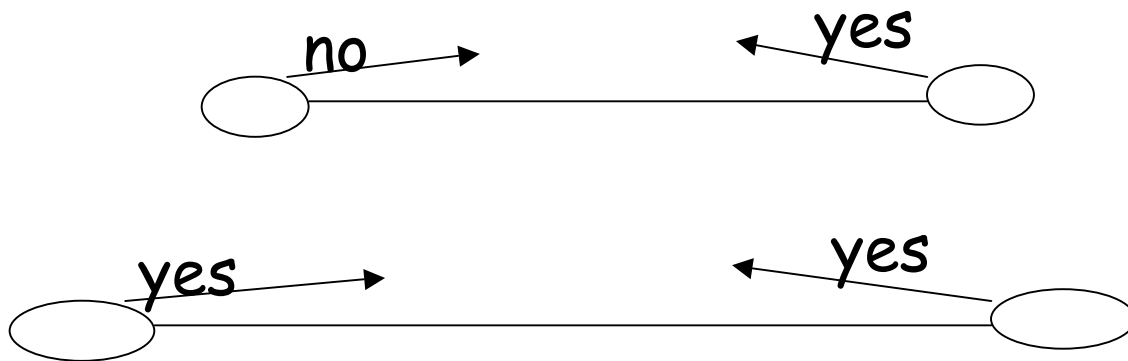
$\text{Messages}(\text{SHOUT}) = 2 M(\text{FLOOD})$

Let's see the details.....

Possible situations



Impossible situations

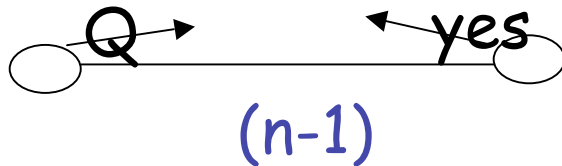


NO/YES msgs are sent to **father** only!

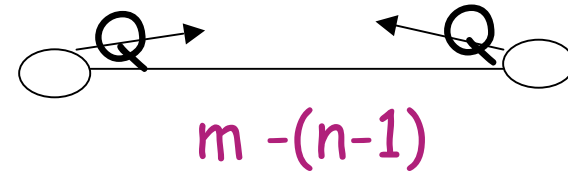
Q is not sent to **father**!

Message Complexity - worst case

Total # of Q msgs:



only one Q on the **ST** links



on the other links

Total: $2(m - (n-1)) + (n-1)$
 $= 2m - n + 1$

Message Complexity - worst case

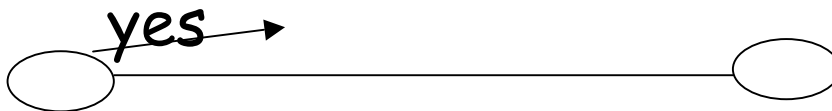
Total # of NO:



as many as Q---Q

$$2(m - (n-1))$$

Total # of YES:



Exactly: $(n-1)$

Message Complexity - worst case

$$\begin{aligned} & 2m - n + 1 + 2(m - (n-1)) + n - 1 \\ &= 2m - n + 1 + 2m - 2n + 2 + n - 1 \\ &= 4m - 2n + 2 \end{aligned}$$

$$\text{Messages}(\text{SHOUT}) = 4m - 2n + 2$$

In fact: $M(\text{SHOUT}) = 2 M(\text{FLOOD}) = 2(2m - n + 1)$

$\Omega(m)$ is a lower bound also in this case (Exercise 3)

Spanning Tree Construction

Without "NO"

Protocol SHOUT+

Is the **NO**-msg really necessary? Why?

- a node needs to know when he can terminate.

Observation. If **y** sends **NO** to **x** THEN it must be:

y has also sent **Q?** to **x** BEFORE!!!!

So, receiving **Q?** can be *interpreted* as **NO**
from the same port!!

States $S = \{\text{INITIATOR}, \text{IDLE}, \text{ACTIVE}, \text{DONE}\}$

Sinit = {INITIATOR, IDLE}

Sterm = {DONE}

INITIATOR

Spontaneously

root := true

Tree-neighbours := { }

send(Q) to N(x)

counter := 0

become ACTIVE

IDLE

receiving(Q)

root := false

parent := sender

Tree-neighbours := {sender}

send(yes) to sender

counter := 1

if counter = |N(x)| then

become DONE

else

send(Q) to N(x) - {sender}

become ACTIVE

ACTIVE

receiving(Q) (to be interpreted as NO)

```
counter := counter + 1  
if counter = |N(x)|  
    become DONE
```

receiving(yes)

```
Tree-neighbours :=  
    Tree-neighbours  $\cup$  {sender}  
counter := counter + 1  
if counter = |N(x)|  
    become DONE
```

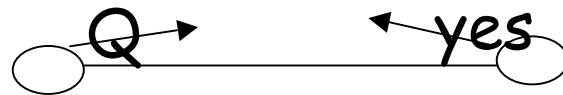
Message Complexity

On each link there will be exactly 2 messages:



either

or



$$\text{Messages}(\text{SHOUT+}) = 2m$$

better than:

$$\text{Messages}(\text{SHOUT}) = 4m - 2n + 2$$

EXERCISE (Global Termination).

Every node knows when he can stop working
(Local Termination)

Can any node decide **when** all nodes have
terminated (Global Termination)???

SPT Construction by Traversal (DFS).

DFS yields a SPT: just "remove" back-edges and get the Return-links (childrens) + the T-links (father).

Distributed implementation:

modify DFT+ (Excercise)

SPT Construction by Broadcast.

THM. The *execution* of every Broadcast Protocol constructs a **SPT**.

Proof's Idea.

Def. **Father(x)** = [the node that **first** sends the **Init-msg** to x]

Fact. the partial order **Father(x) > x** defines a **SPT** rooted in the Initiator.

Obs. Just a Broadcast does not suffice!!!!
(**No** local knowledge of the **SPT**)

SPT: FINAL CONSIDERATIONS

- The computed SPT **depends** on the **Protocol**.
- The computed SPT **depends** on the execution of the **Prot**.

Example. Protocol **Shout (+)**.

Every possible SPT of **G** could be the output of **Shout**

“ Just manage the msg delays ”

Consequence. $\text{diam}(\text{SPT}) \gg \text{diam}(G)$ (Bad news)

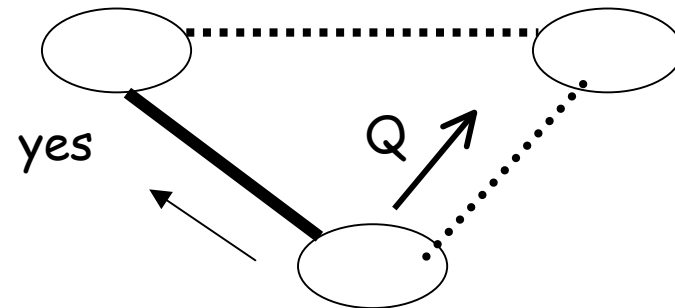
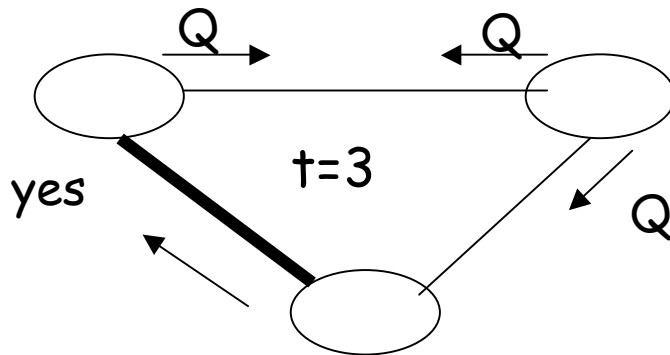
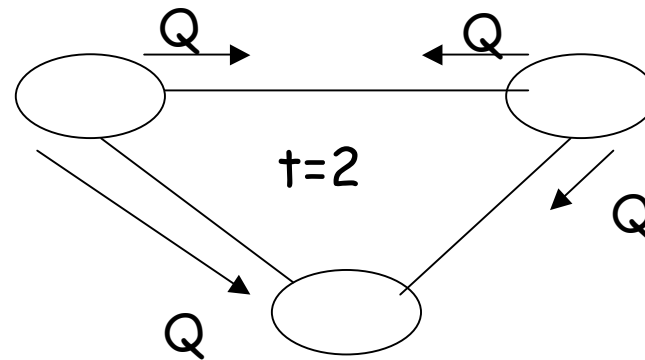
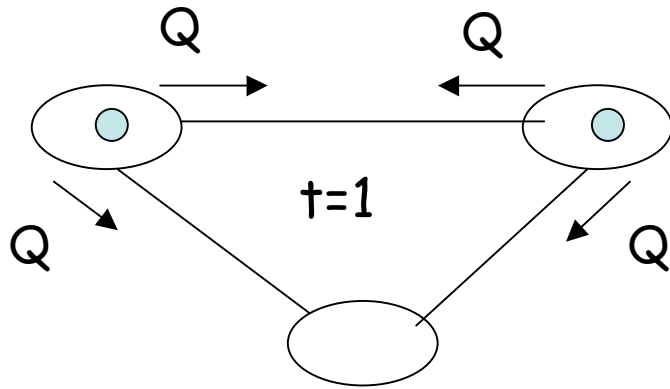
New Problem. Compute SPT T^* s.t.

$$\text{diam}(T^*) \approx \text{diam}(G)$$

SPT Construction with Multiple Initiators

- * What about Prot. **Shout (+)** for that ?
- ** Can we design new **deterministic** protocols for this problem?

Multiple Initiators

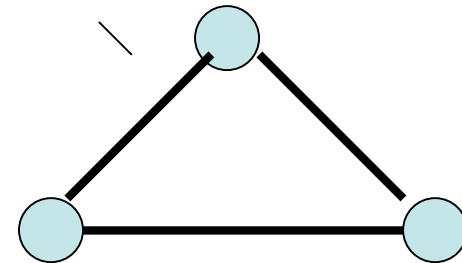


Not Correct !!

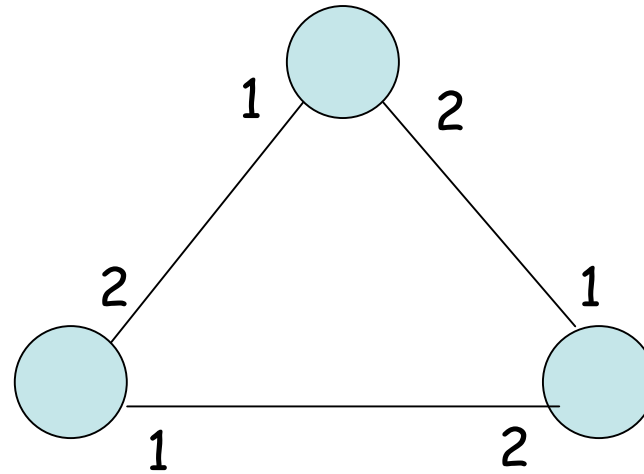
THM. The SPT Problem is *deterministically unsolvable* under **R**

Proof . * 3 identical connected nodes x,y,z having the same initial local states (**All Initiators**).
* Synchronous RUN.
* All start at $t=0$.

Fact. At every $t > 0$, they must
- be in the same state,
- do the same things



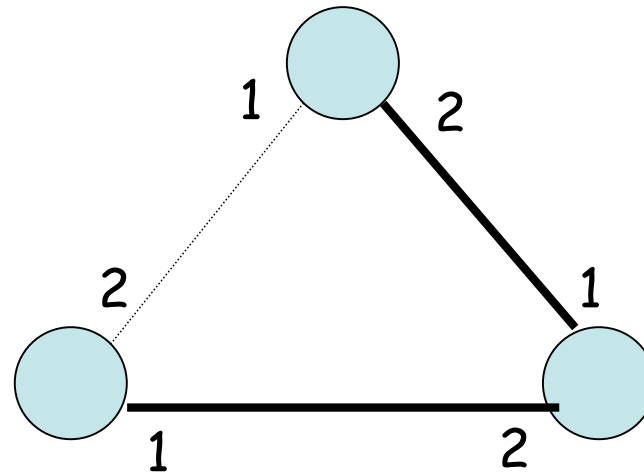
So, it will **impossible** for them to select 2 links over 3



Starting
Configuration



Nodes should
behave **differently**
and....get



Final
Configuration

An election is needed to have a unique initiator.

or

Another protocol has to be devised.

NOTE: Election is impossible if the nodes do not have distinct IDs

Depth First Traversal

Assumptions

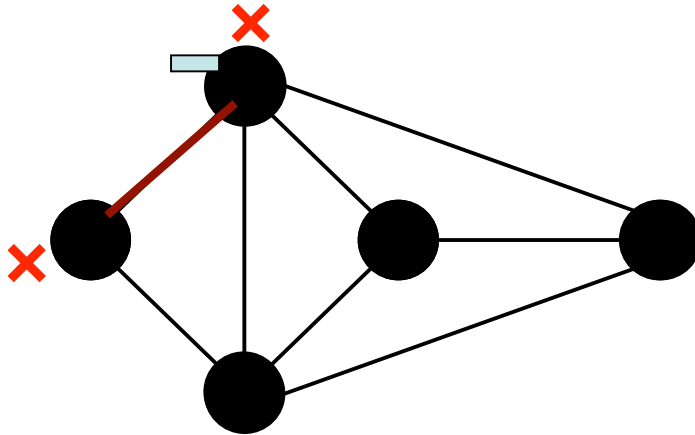
- Single initiator
- Bidirectional links
- No faults
- G connected

$S = \{\text{INITIATOR}, \text{SLEEPING}, \text{ACTIVE}, \text{DONE}\}$

DFT - Protocol. First version

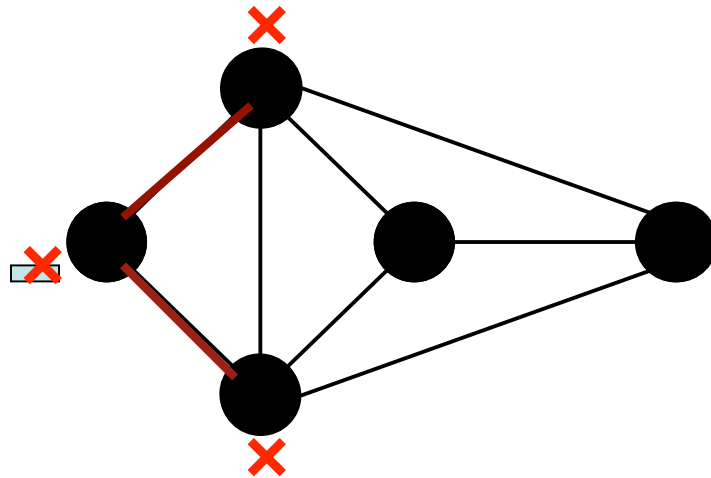
- 1) When **first** visited, - remember who sent (**father**)
 - forward the **token** to one of the unvisited neighbours
 - wait for its reply
- 2) When (neighbor) receives **token**,
 - *if already visited*, it will return the **token** to **father** saying it is a **back edge**
otherwise, will forward it (sequentially) to **all** its unvisited neighbours before returning it
- 3) If there are no more unvisited neighbours, return the token (**reply**) to **father**
- 4) Upon reception of **reply**, forward the **token** to another unvisited neighbour

How it works...



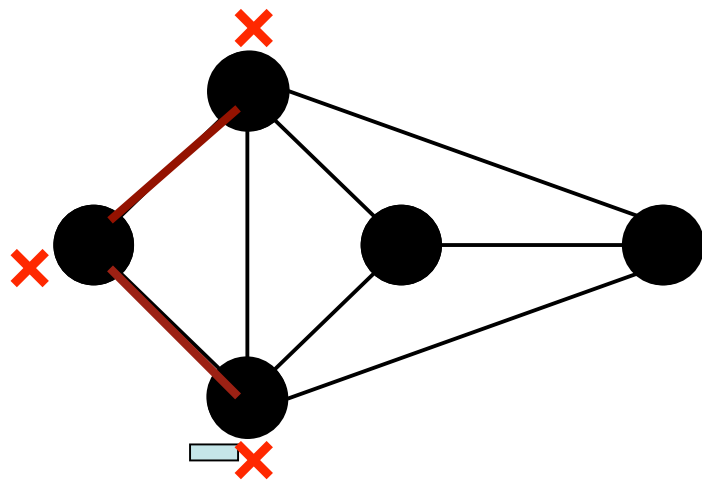
DFS edges: 

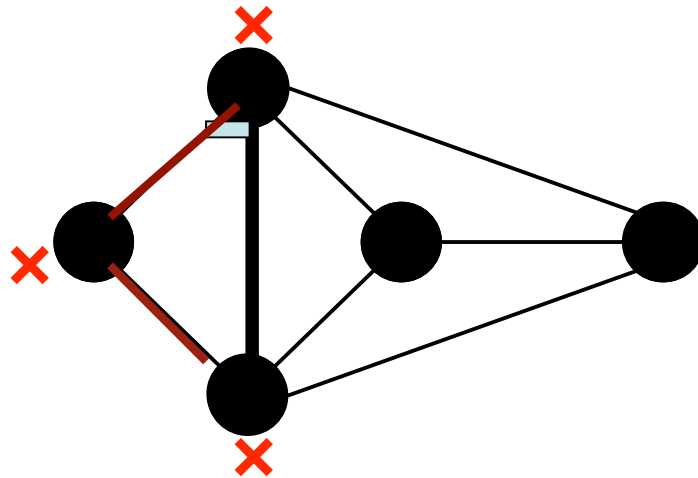
Backedges: 



Used edges: 

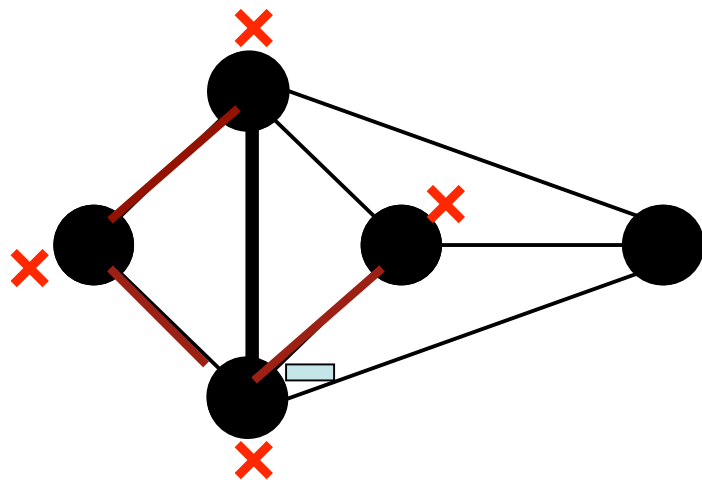
Backedges: 

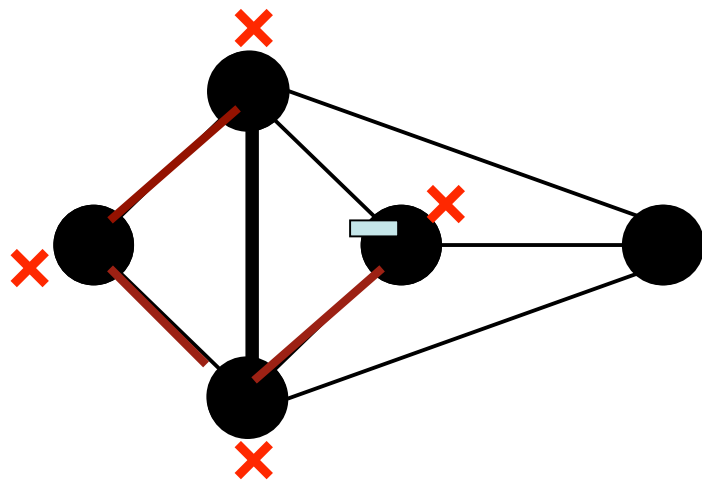


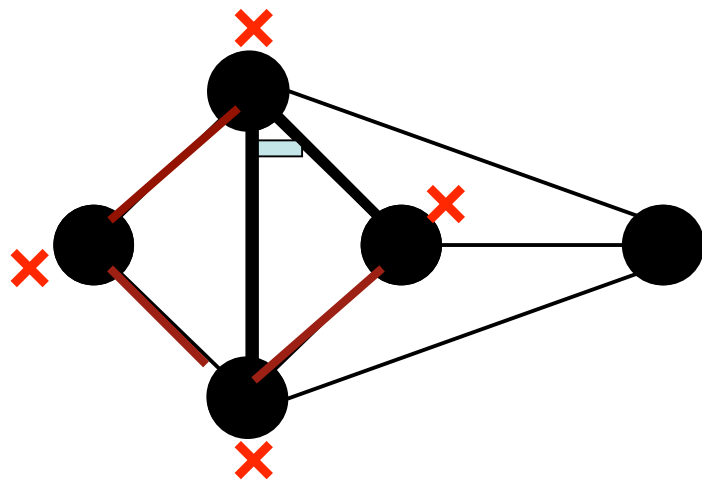


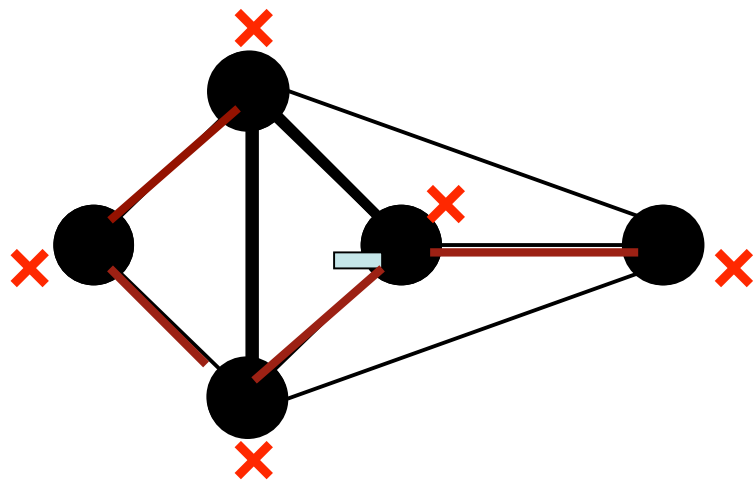
Used edges: 

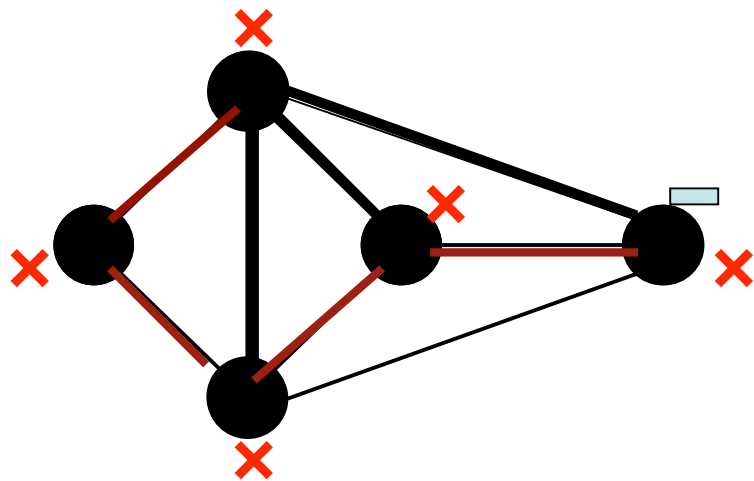
Backedges: 

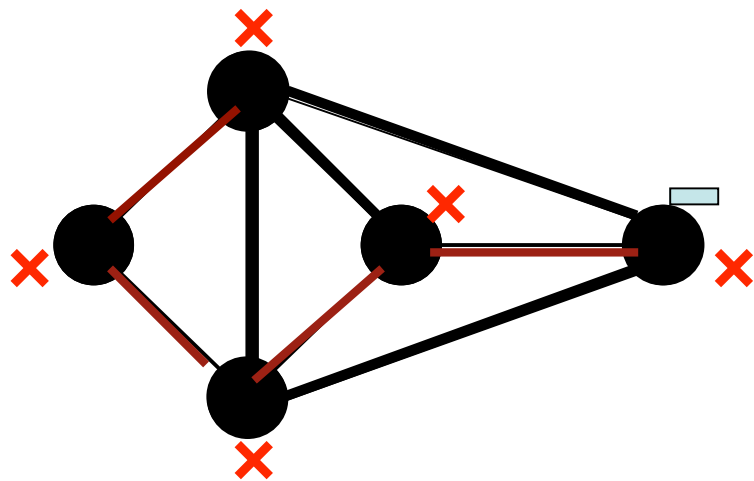


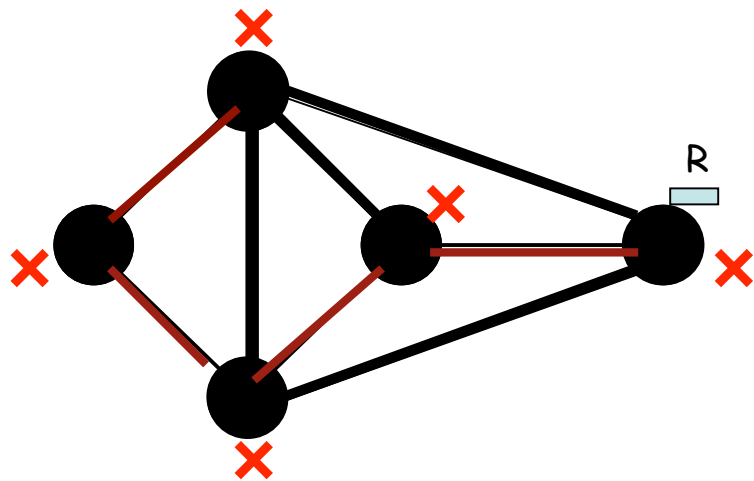


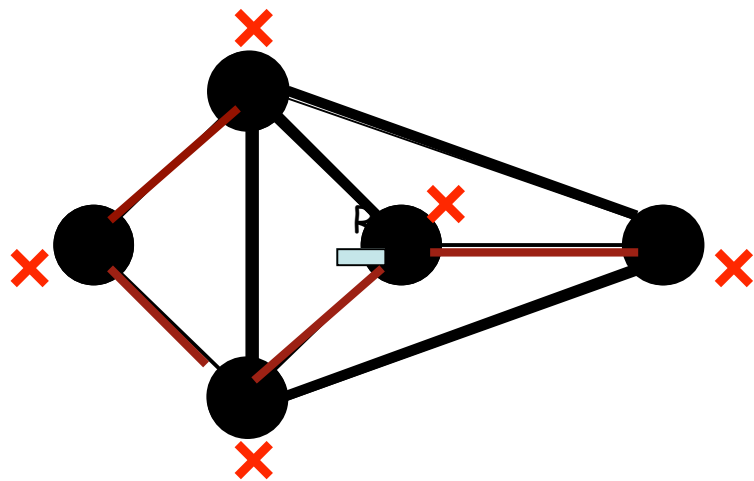












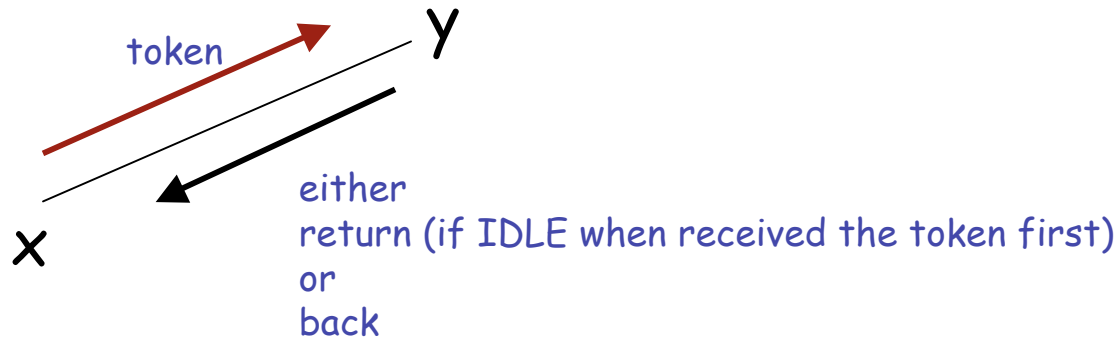
Protocol Back

- 1) When **first** visited, - remember who sent (**father**)
 - forward the **token** to one of the unvisited neighbours
 - wait for its reply
- 2) When (neighbor) receives **token**,
 - *if already visited*, it will return the **token** to **father** saying it is a **back edge**
otherwise, will forward it (sequentially) to **all** its unvisited neighbours before returning it
- 3) If there are no more unvisited neighbours, return the token (**reply**) to **father**
- 4) Upon reception of **reply**, forward the **token** to another unvisited neighbour

Traversal Complexity

Message Complexity:

Type of messages: token, back, return



$$M(\text{Back}) = 2m = O(m)$$

$\Omega(m)$ is also a lower bound
for DFT

Time Complexity (Ideal Time)

DFT is **FULLY SEQUENTIAL!** so,

$$\text{Time(Back)} = M(\text{DFS}) = \Theta(m)$$

THM. General Lower bound.

$$\text{Time(DFT)} = \Omega(n)$$

There may be a big gap....

Obs. in dense graphs,
most msg may be on **Back Edges**

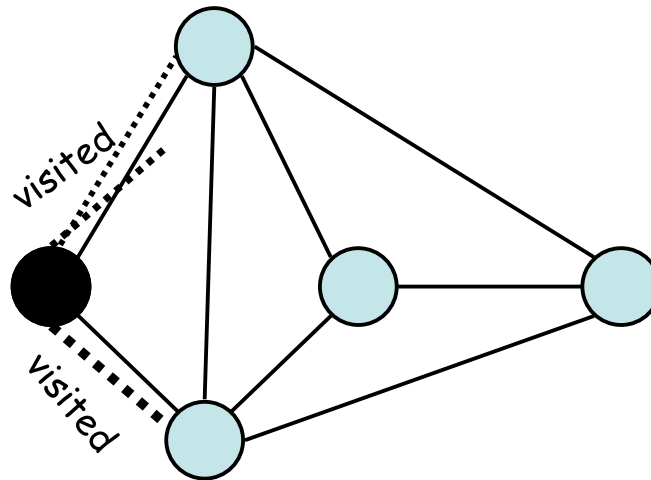
---> most time is spent on **Back Edges**

Challenge: avoid sending messages on **Back Edges**

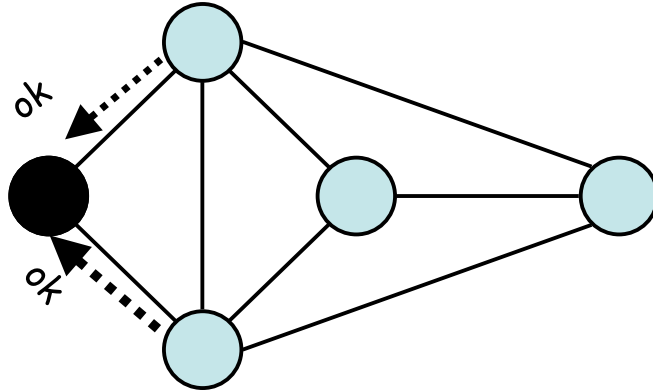
How ?

DFT+ (Improving Time)

Idea: x informs **in** $N(x)$ when it is visited



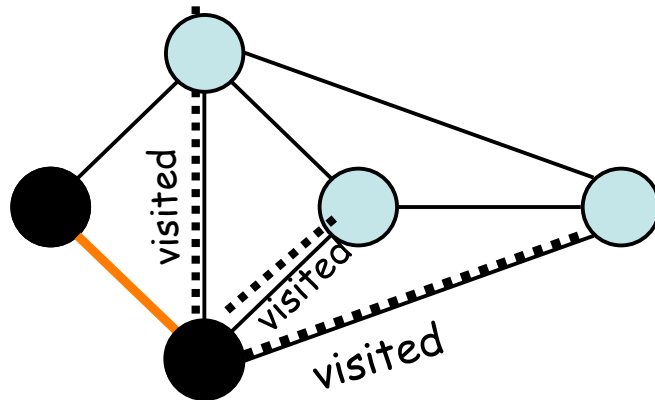
DFT+ Improving Time



$N(x)$ sends **ack** *OK* back in **||**.

This is necessary when the system is not synchronous

DFT+ Improving Time

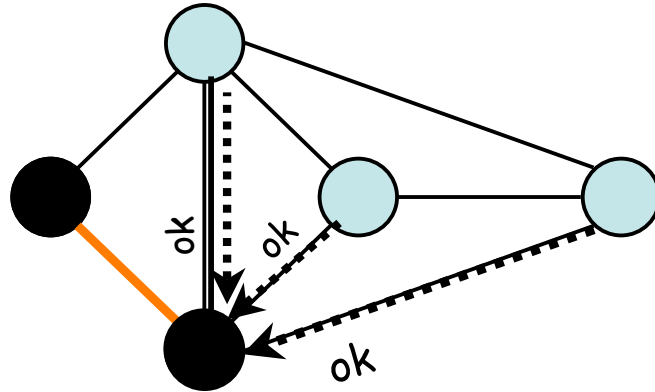


After receiving all acks,

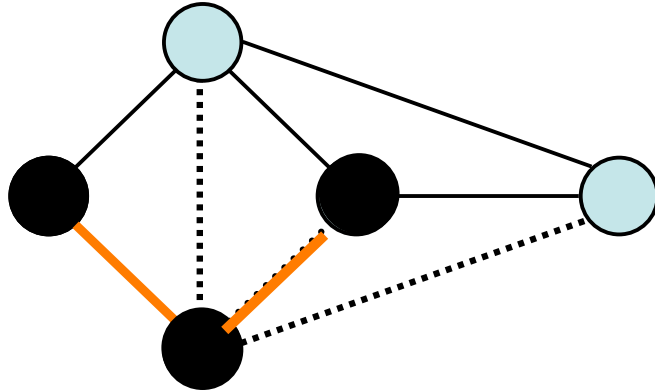
x sends the token to one of its **unvisited** neighbors



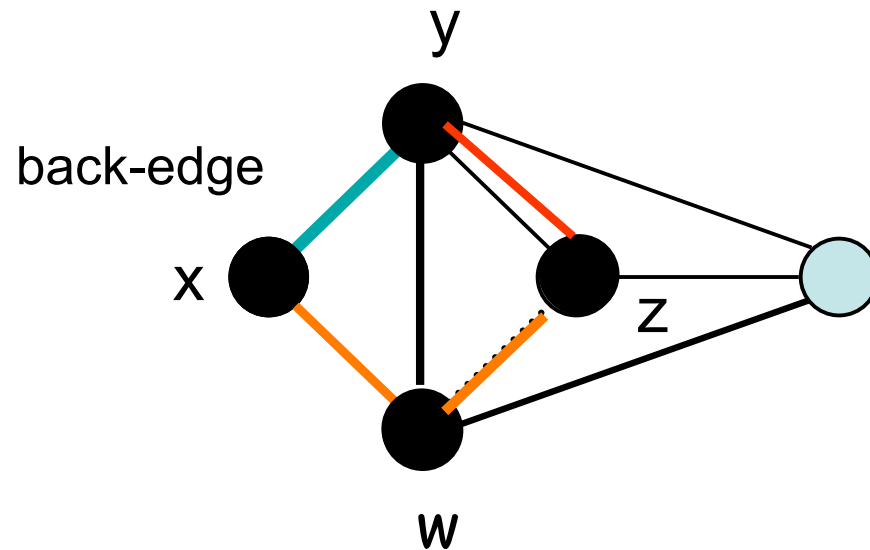
x already knows all of them!



DFT+ Improving Time



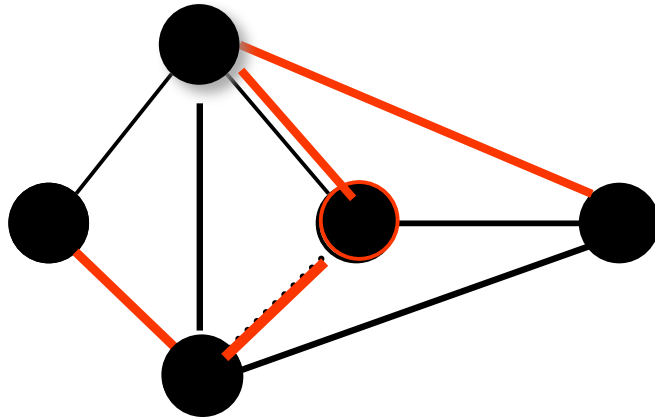
DFT+ Improving Time



y knows that **x,w** are visited and **z** is his father.

So, back-edges are discovered **in parallel**.

DFT+ Improving Time



DFT+ Complexity

Message

Messages: Token, Return, Visited, Ack (ok)

Each entity (except init): receives 1 Token, sends 1 Return:

$$2(n-1)$$

Each entity:

1 Visited to all neighbours except the sender

Let s be
the initiator

$$\begin{aligned} & |N(s)| + \sum_{x \neq s} (|N(x)| - 1) \\ &= 2m - (n-1) \end{aligned}$$

- same for Ack: $2m - (n-1)$

$$\text{TOT: } 4m - 2(n-1) + 2(n-1) = 4m$$

DFT+ Complexity

Time (ideal time)

If the system is synchronous:

Token and Return are sent **sequentially**: $2(n-1)$

Visited and Ack are done **in parallel**: $2n$

TOT: $4n - 2$

The time is linear in **n** and not in **m** !

Summarizing:

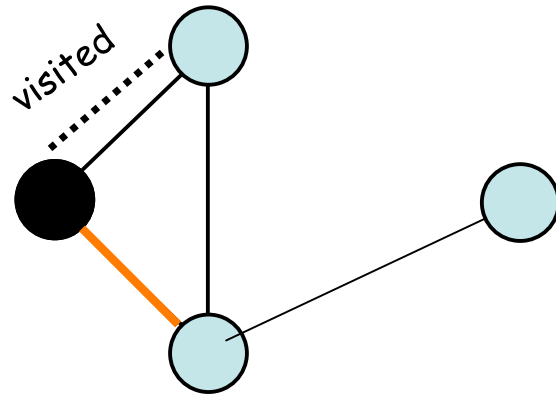
DF Traversal

	Messages	Ideal Time
DFT:	$2m$	$2m$
DFT+:	$4m$	$4n - 2$

DF++

Do not send the Ack
What happens ?

EXCERCISE !!!



Computations with Multiple initiator: WAKE-UP

COMPLETE GRAPH

Broadcast

Flood		Specific
$O(n^2)$		$O(n)$

Wakeup

Flood	Specific
$\Omega(n^2)$	
Need additional assumptions to reduce the complexity	

HYPERCUBE

Broadcast

Flood		Specific
$O(n \log n)$		$O(n)$

Wakeup

Flood	Specific
$\Omega(n \log n)$	